

论文：《Patch n' Pack: NaViT, a Vision Transformer for any Aspect Ratio and Resolution》

论文链接：<https://arxiv.org/abs/2307.06304>

代码仓库（非官方）：<https://github.com/kyegomez/NaViT>

1. 背景

之前的Vision Transformer(ViT)存在两个问题（改进点）：

- 输入ViT的图片需要做缩放(resize)处理，使得图片最终能处理成一个定长的序列表征。
- ViT本身支持变长序列的modeling，但是需要进行packing提高训练效率。

NaViT (Native Resolution ViT) 为了解决上面的问题分别使用了：

- Arbitrary resolutions and aspect ratios：保留原始分辨率和宽高比。
- Sequence packing during training：将多个图片按原始的分辨率做Patch处理后，然后拼接到一个序列中，来实现对不同分辨率图像可统一处理，[论文](#)称这种方法为Patch n' Pack。

然后最直观的效果就是在相同的计算成本下达到更好的效果：

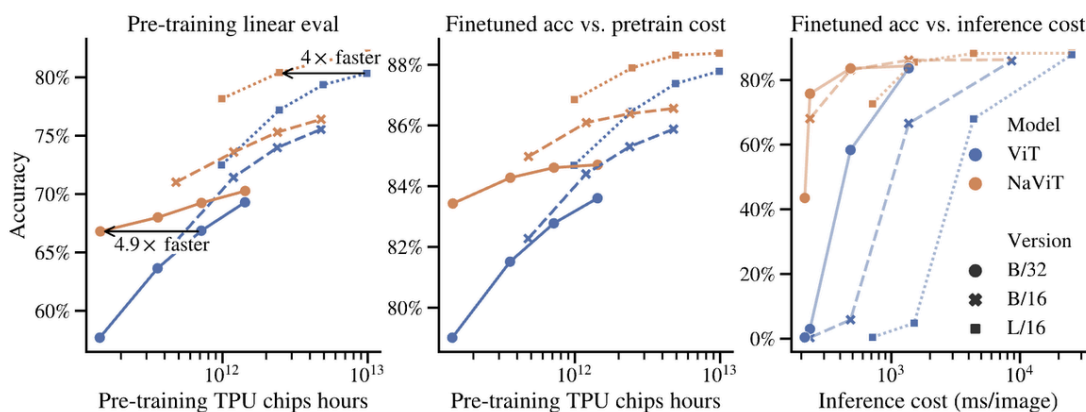


Figure 1: NaViT offers notable computational efficiency during pre-training (left) which carries over to downstream fine-tuning (middle). A single NaViT can be applied successfully to multiple resolutions (right), smoothly trading off performance and inference cost.

2. NaViT (Native Resolution ViT)

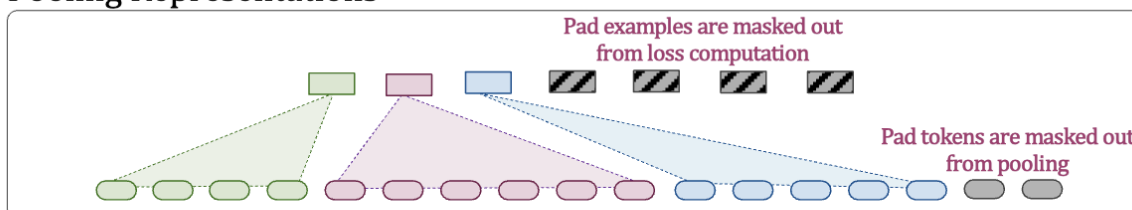
2.1 Architectural changes

- 掩码注意力和掩码池化机制 Masked self attention and masked pooling

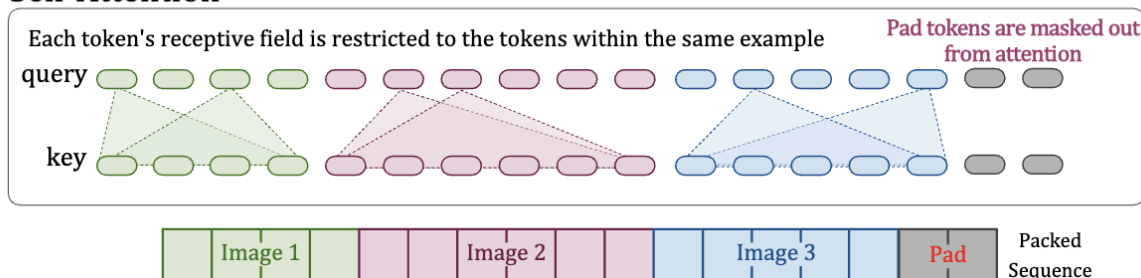
多example打包到一个序列后，在模型前向计算时，Transformer的每一层都会做Self-Attention计算，为了防止不同example相互attention，引入了一个额外的自注意力的掩码。此外通常CV建模任务中，需要对图像所有表征做pooling来计算loss，同样需要做计算隔离，因此也需要跟Self-Attention计算一样的掩码矩阵。

如图所示，一个序列有三个example，token长度分别为：4, 6, 5。最后为了对齐Batch的多序列拼接了2个padding token：

Pooling Representations



Self-Attention



这个额外的Attention Mask其实就是一个对角分块Mask矩阵。

- **分解&分数位置编码** Factorized & fractional positional embeddings

(1) ViT用的是一个可训练的1D绝对位置编码。

(2) [Pix2struct](#) 使用的是一个可学习的 2D absolute positional embedding，其大小为 $[\max_{l \in n}, \max_{l \in n}]$ ，每个位置 (x, y) 都要被学习过，才能获得较好的效果。

(3) NaViT使用的分离式的位置编码对 x 轴和 y 轴的参数分别进行学习，一方面参数量大大减少，由 $\max L \in n^2$ 减少到 $2 \times \operatorname{maxLen}$ ，另一方面分离式的位置编码，对未见过的分辨率有更好的外推性，因为虽然 (a, b) 组合的分辨率没见过，但高是 a 或宽是 b 的图片模型可能分别都见过，这就能学到对应的位置表征。

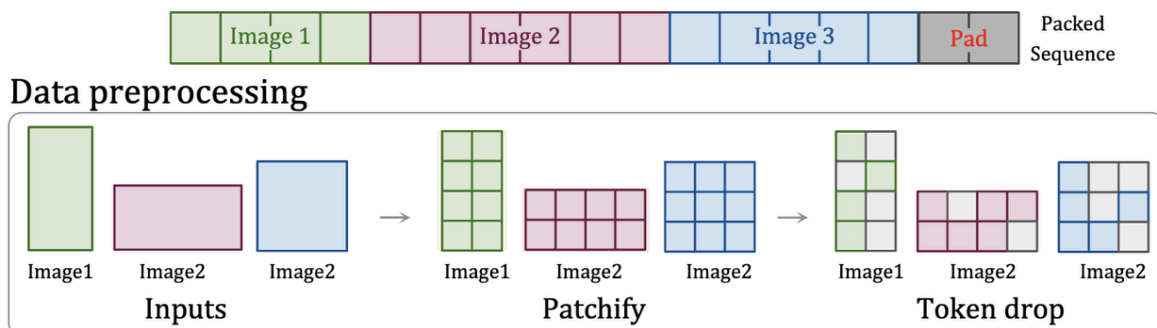
2.2 Training changes

- **Continuous Token dropping**

图像本身是低密度的信息载体，对于空间临近的一些像素和Patch会有较相似的信息，因此在模型训练时，可以通过适当的采样，丢弃一些Patch以提高训练吞吐和性能。

对于传统的ViT方法对每个固定分辨率图片只能采用固定drop rate来采样，将每个图片处理成定长的序列以输入给模型。NaViT由于多个image被pack到一个序列里，每个图

片可以有不同的drop token的策略，这为不同分辨率的图片提供了灵活的token采样方式，尤其对于一些大尺寸的图片，通过提升drop rate来压缩序列特征长度；同时为了控制Sequence的总长度，可以对最后一个图片做特化的采样策略（如：根据剩余可容纳几个token位置，来设置采样策略），来控制多image拼接后得到一个固定sequence长度，这大大简化了Batch的处理逻辑。



不同图片的预处理过程如图所示，对多图片保留真实的分辨率和宽高比做Patch分块处理，然后对每个图片做不同的drop Token处理，最后拼接成一个序列。

• Resolution sampling

NaViT 可以在保持纵横比的同时对图像的像素总数重新采样。

对于传统的 ViT 而言，始终存在这样一种权衡：使用较小图像训练可以获得更高的吞吐量（即更快训练），但使用较大图像训练则在推断阶段拥有更高分辨率，因而可能带来更好的模型性能。因此，通常会先以较小分辨率进行预训练，然后在较高分辨率下进行微调。

而 NaViT 具有更高的灵活性：它允许对图像尺寸进行混合分辨率采样，同时仍保留每张图像的原始纵横比。这样一来，就能兼顾更高的吞吐量与对大尺寸图像的训练，从而在相同的模型规模与相同的训练时长下，显著提升相对于传统 ViT 的性能。

• Padding examples

通过Pack后的序列再组Batch，一个Batch中每个序列包含的example的数量是不同的。NaViT 做了Batch内example对齐处理：

对于一个Batch为 B 的序列，我们最多提取 $E_{\max}$ 个pooling后的表征，也就是每个序列都提取 $E_{\max}$ 个图像Pooling的表征。如果一个序列包括超过 $E_{\max}$ 的图像，超过的部分直接丢弃；如果一个序列的examples少于 $E_{\max}$ ，使用 fake 表征对序列做padding。

3. Experiments

与传统的ViT相对，达到传统ViT的最佳性能，只需要更少的计算开销；另外如果使用相同的计算资源，NaViT的性能始终优于ViT。

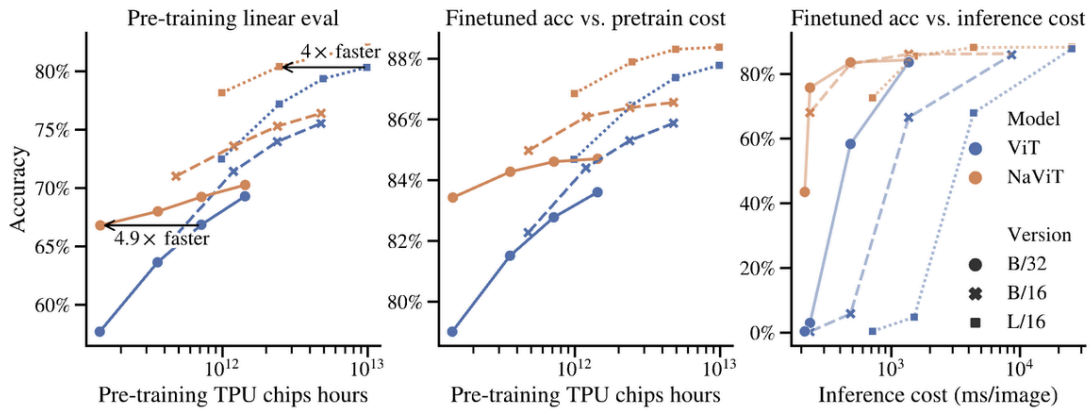


Figure 1: NaViT offers notable computational efficiency during pre-training (left) which carries over to downstream fine-tuning (middle). A single NaViT can be applied successfully to multiple resolutions (right), smoothly trading off performance and inference cost.

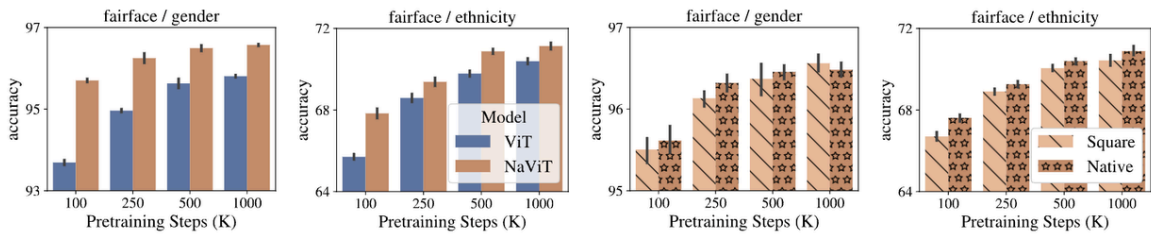


Figure 12: Evaluating the accuracy of annotators trained on fairness-related signals using either NaViT-L/16 or ViT-L/16: **Left:** NaViT offers better representations that improve the accuracy of annotators. **Right:** Using native aspect ratios in NaViT results in a higher performance when compared to resizing images to squares.